Flight Data Analysis with Apache Spark

Nicola Violante
dept. Computer Engineering
University of Rome Tor Vergata
Rome, Italy
nicola.violante@alumni.uniroma2.eu

***Abstract*—This project presents the development of an end-to-end pipeline for the batch analysis of historical flight data. Operating within a fully containerized environment, the architecture integrates specific tools to manage the entire data lifecycle: Apache NiFi [1] for ingestion, Hadoop HDFS [2] for distributed storage, and Apache Spark [3] for processing. The serving layer relies on InfluxDB [4] and Redis [5], while the visualization of metrics is delegated to Grafana [6]. The primary objective was to implement two analytical queries on the reference dataset, alongside a comprehensive performance evaluation. This study compares the efficiency of different Spark APIs (RDD, DataFrame, SQL) and the impact of storage formats (CSV, Parquet) under varying degrees of horizontal scalability.**

I. Introduction

This project aims to utilize the distributed computing framework Apache Spark to analyze historical flight data provided by the United States Department of Transportation [7]. Specifically, a preprocessed version of the original dataset was employed, comprising approximately 2200000 events related to flights operated within the United States between January 1st and April 30th, 2025. In addition to developing the analytical queries, this work includes a comprehensive evaluation of system performance, measuring execution times across different Spark APIs (RDD, DataFrame, SQL), data storage formats, and varying degrees of parallelism.

II. System Architecture

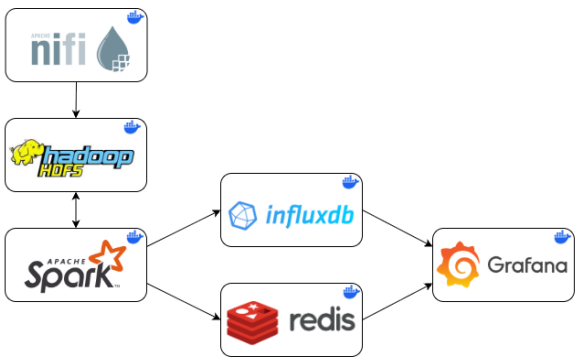


Fig. 1. System Architecture

The proposed infrastructure is designed as a fully containerized ecosystem orchestrated via Docker Compose [8], ensuring portability, isolation, and ease of deployment. The architecture is modular and logically divided into five distinct layers, each handled by a dedicated Big Data framework:

A. Ingestion Layer (Apache NiFi)

Serving as the entry point of the pipeline, NiFi automates the retrieval of the historical flight data. It handles the downloading, decompression, data cleansing, and routing of the dataset into the distributed storage system

B. Storage Layer (HDFS)

A distributed file system is employed as the primary data lake. The cluster is configured with one master node and three worker nodes to guarantee data replication, fault tolerance and high availability. It stores the preprocessed datasets in both textual (CSV) and columnar (Parquet) formats to support subsequent benchmarking.

C. Processing Layer (Apache Spark)

The computational core of the architecture relies on a standalone Spark cluster. It consists of a Spark Master and a scalable number of Spark Worker nodes. The workers are constrained in terms of resources (cores and memory) to allow for controlled and reproducible performance benchmarking across different degrees of horizontal parallelism.

D. Serving Layer (InfluxDB & Redis)

To efficiently serve the processed analytical results to the visualization layer, a dual-database strategy was adopted based on the nature of the queries:

1) InfluxDB: A time-series database utilized to store the temporal aggregations generated by Query 1 (e.g., monthly delays) as well as the execution time metrics collected during the benchmark runs.

2) Redis: An in-memory data store used to save the static, rank-based results of Query 2, guaranteeing extremely low-latency access for top-N leaderboards.

E. Visualization Layer (Grafana)

The front-end observability platform connects simultaneously to both InfluxDB and Redis. It hosts interactive dashboards to monitor flight cancellation and delay trends, as well as benchmark metrics. Additionally, a Grafana Image Renderer container is integrated to allow the seamless export of panel visualizations.

III. Data Ingestion and Storage

The data ingestion phase is fully automated and orchestrated using Apache NiFi, which manages the entire lifecycle of the dataset from the remote source to the distributed file system. To ensure seamless deployment and avoid manual intervention via the Web UI, the NiFi flow is programmatically controlled using its native REST APIs. A dedicated shell script called *run_nifi.sh* handles authentication via JWT token, enables the necessary services, dynamically discovers the processor IDs, and triggers the flow execution.

A. NiFi Data Flow

The ingestion pipeline is composed of a sequence of specialized processors designed to extract, transform, and load (ETL) the data efficiently. To ensure data idempotency, meaning the pipeline can be executed multiple times without duplicating records, the flow is explicitly designed to overwrite existing datasets on HDFS.

1) **InvokeHTTP:** Initiates the workflow by fetching the raw dataset directly from the provided remote URL as a .tar.gz archive.
2) **CompressContent & UnpackContent:** These processors work in tandem to first decompress the GZIP file and then extract the underlying TAR archive, generating individual FlowFiles for each extracted file.
3) **RouteOnAttribute:** Applies a filtering mechanism using the NiFi Expression Language to discard unwanted items (e.g., macOS hidden ._ files) and retain exclusively the relevant .CSV reporting files.
4) **QueryRecord:** Performs early data reduction (schema-on-read). By executing a SQL query directly on the incoming FlowFiles, it extracts only the required features (e.g., OP_UNIQUE_CARRIER, DEP_DELAY, CANCELLED, etc.), discarding unnecessary columns and significantly reducing the data footprint before it reaches the cluster. Additionally, its underlying *CSVReader* and *CSVRecordSetWriter* controller services are intentionally configured to skip and strip the original headers from the individual files, standardizing the data for the upcoming merging phase.
5) **MergeContent:** This is a crucial step for storage optimization. It aggregates the multiple filtered, headerless FlowFiles into a single, cohesive dataset using a bin-packing algorithm. This aggregation prevents the ”small files problem” in Hadoop, which would otherwise lead to underutilized 128 MB HDFS blocks and severe NameNode memory overhead.
6) **ReplaceText:** Ensures data consistency by prepending a uniform, predefined CSV header to the merged content.
7) **UpdateAttribute:** Replaces the dynamically generated FlowFile name with a static identifier (e.g., dataset) to guarantee idempotency; by standardizing the filename, downstream processors will target the exact same file path during subsequent executions.
8) **PutHDFS & PutParquet:** The pipeline branches to persist the final dataset into HDFS in two distinct formats. *PutHDFS* writes the standard text-based CSV format, while *PutParquet* converts and stores the data in a highly optimized columnar format. Both processors are configured with overwrite/replace conflict resolution strategies which, combined with the static filename, ensure that old data is safely overwritten.
9) **LogMessage:** Acts as an automated error-handling mechanism. It is routed to the failure relationships of the storage processors, logging a critical error message if the HDFS cluster becomes unreachable or if the data persistence fails.

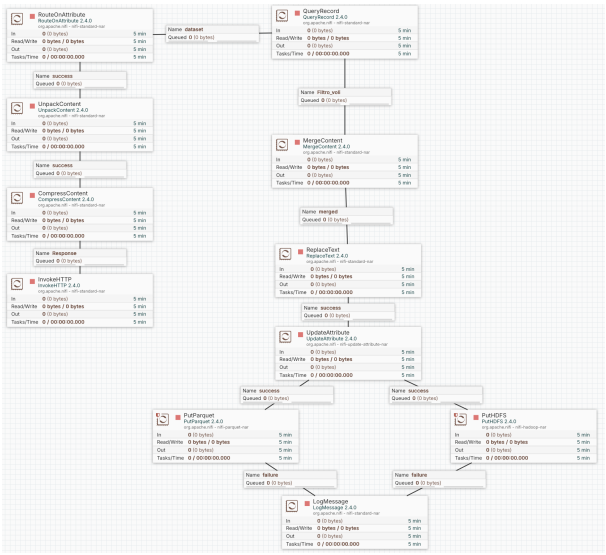


Fig. 2. NiFi flow

B. Storage Strategy and Volume Analysis

Storing the dataset in both CSV and Parquet format is a deliberate architectural choice intended to evaluate the performance differences between row-based and column-based storage engines during the Spark benchmarking phase. The transformation applied by NiFi yields a significant reduction in data volume. The initial compressed archive has a size of approximately 51.75 MB, which expands to roughly 244.17 MB across multiple uncompressed CSV files. Following the column filtering and aggregation, the optimized CSV dataset is reduced to 77.18 MB. The conversion to Parquet format further maximizes storage efficiency through its internal dictionary encoding and compression, resulting in a final footprint of only 9.01 MB.

IV. Data Processing

The core analytical engine of the architecture is Apache Spark. The processing logic was developed to be highly modular, allowing the execution of identical analytical queries using three different Spark APIs: RDDs, DataFrames and Spark SQL.

A. Application Design and Software Modularity

To ensure maintainability, scalability, and adherence to the DRY principle, the Spark application was designed with a highly modular, decoupled architecture.

- **Dynamic Routing:** The execution is orchestrated by centralized entry points. These scripts utilize command-line arguments and Python's *importlib* library to dynamically import and route the execution to the appropriate query module at runtime.
- **Separation of Concerns:** A strict decoupling between I/O operations and analytical business logic was enforced. The individual query files are strictly confined to data processing. They expose a core logic function that receives a normalized DataFrame, applies the transformations, and returns a new DataFrame, remaining completely agnostic of the data's origin or destination.
- **Utility Abstraction:** All boilerplate operations, such as SparkSession initialization, HDFS availability checks, schema enforcement, data casting, and the complex exporting logic to HDFS, InfluxDB, and Redis, are abstracted away into dedicated helper modules. This architectural choice keeps the analytical scripts exceptionally clean and guarantees that the benchmark runs evaluate exclusively the algorithmic efficiency of the Spark engines, untainted by I/O overhead.

B. Execution Model and Optimizations

The Spark cluster operates in standalone mode. To ensure consistent performance testing, the SparkSession is dynamically initialized and stopped for each query execution, thereby preventing cross-contamination of cached data in RAM. A key aspect of Spark's execution model is lazy evaluation: transformations are recorded in a Directed Acyclic Graph and are not executed until an action is triggered. While the RDD API provides low-level, fine-grained control over the data distribution, it lacks built-in optimization. Conversely, the DataFrame and Spark SQL implementations heavily leverage the Catalyst Optimizer, which automatically generates optimized physical execution plans, significantly reducing execution times.

C. Query 1: Monthly Delay and Cancellation Trends

The first query aims to compute the monthly minimum, maximum and average departure delays, alongside the cancellation rate, specifically for American Airlines (AA) and Delta (DL). A critical requirement is to exclude canceled flights from the delay statistics while retaining them to compute the global cancellation rate. To satisfy this, the query logic was implemented across three different APIs, highlighting distinct execution paradigms.

1) DataFrame API Implementation: The DataFrame API offers a declarative approach, heavily optimized by Spark's internal engine.

- **Pre-Aggregation Filtering and Pushdown:** The process begins with a narrow transformation (*df_filtered = df.filter(F.col(CARRIER).isin("AA", "DL"))*). Thanks to the Catalyst Optimizer, this predicate is "pushed down" as close to the data source as possible, minimizing the volume of data loaded into memory.
- **Conditional Aggregation:** A wide transformation is triggered by *groupBy(MONTH, CARRIER)*. To handle the canceled flights condition dynamically, the *F.when()* construct is utilized within the *.agg()* method. This allows Spark to compute both the delay averages (ignoring canceled flights) and the cancellation rate (evaluating all rows) in a single pass over the grouped partitions.
- **Sorting:** Finally, an *orderBy()* operation is applied, triggering a secondary shuffle based on a range partitioner to globally sort the output.

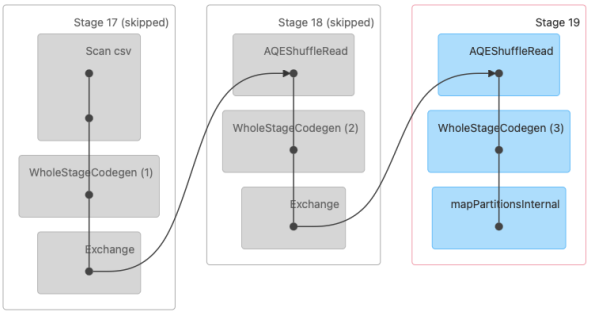


Fig. 3. Query 1 DataFrame

2) Spark SQL API Implementation: The SQL approach registers the underlying DataFrame as a temporary view and executes a purely declarative SQL string. The query logic uses a *WHERE* clause for filtering and *CASE WHEN* statements to dynamically handle the canceled flights during the aggregation phase. Thanks to the Catalyst Optimizer, this SQL string is translated into the exact same physical execution plan as the DataFrame implementation. As a result, this approach achieves perfect execution parity with DataFrames, offering the familiar readability of standard SQL while fully benefiting from the same underlying performance optimizations.

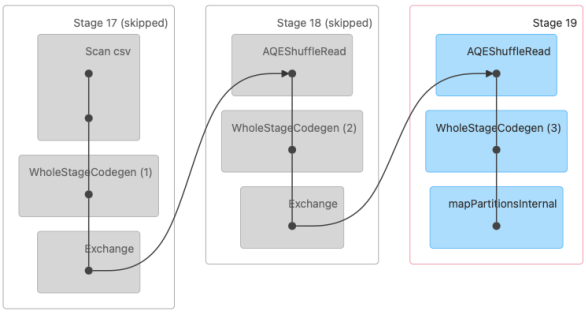


Fig. 4. Query 1 SQL

3) RDD API Implementation: Unlike the higher-level APIs, the RDD implementation represents an imperative paradigm, bypassing the Catalyst Optimizer and requiring manual pipeline optimization.

- **Row Parsing and Map-Side Filtering:** The process begins with a *flatMap(parse_row)*. This custom function

performs a combined narrow transformation: it manually filters out non-target carriers and malformed rows, and maps the valid records into complex key-value pairs: *((month, carrier), (delay_sum, count_non_canc, delay_min, delay_max, count_canc))*.

- **Distributed Aggregation and Map-Side Combine:** The core wide transformation is achieved via *reduceByKey()*. A significant advantage is that it natively performs a map-side combine. It partially aggregates the delay sums and counts on the local executor nodes before transmitting the data across the network. This drastically reduces the network I/O during the shuffle phase.
- **Finalization:** A subsequent narrow *map(format_results)* is applied to the reduced RDD to calculate the final averages and percentages, followed by a *sortBy()* transformation to yield the ordered output.

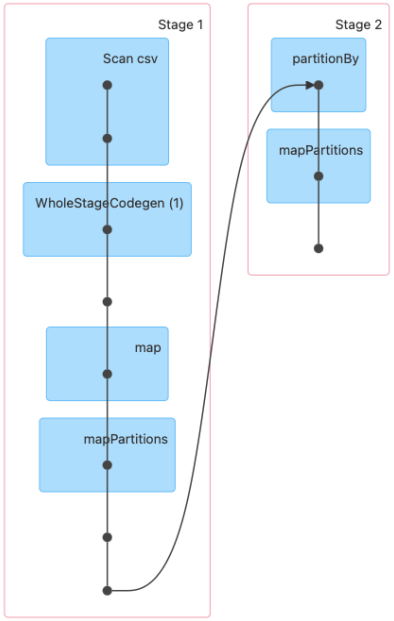


Fig. 5. Query 1 RDD

D. Query 2: Carrier Ranking and Delay Causes

The second analytical query is designed to evaluate the operational performance of individual airlines. It calculates the total number of valid flights, the average arrival delay, and the average contribution of five specific delay components (Carrier, Weather, NAS, Security, and Late Aircraft). The query filters out airlines with fewer than 500 valid flights and returns the top 10 carriers ranked by the highest average arrival delay. In calculating these delay components, missing values (Null) were explicitly treated as zeros. Excluding these records would have restricted the calculation of the average strictly to flights affected by a specific disruption, thereby skewing the analysis. This would have yielded statistically distorted metrics: rare but severe events would have generated disproportionate averages, failing to accurately reflect the airline's overall operational efficiency. By imputing null values as zeros, the impact of each delay is properly distributed across the entire volume of valid flights. It is worth noting that the mathematical sum of these component averages will physiologically exceed the total mean arrival delay. This occurs because the overall arrival delay is significantly lowered by negative values from early flights, for which delay causes are inherently absent. As with the previous analysis, three distinct implementations were developed to highlight the different execution semantics of the Spark APIs.

1) DataFrame API Implementation: The DataFrame approach relies on the declarative nature of the API to allow Spark's Catalyst Optimizer to handle the execution plan.

- **Pre-Aggregation Filtering and Pushdown:** The process begins with a narrow transformation to strictly retain valid flights (*df.filter((F.col(CANCELLED) == 0) & (F.col(DIVERTED) == 0))*). Thanks to the Catalyst Optimizer, this predicate is 'pushed down' early in the execution plan. This guarantees that irrelevant records are discarded at the source, minimizing memory consumption and ensuring they do not participate in the costly shuffle phase.
- **Global Aggregation:** A *groupBy(CARRIER)* triggers a wide transformation. The *.agg()* function computes the row count alongside the distinct averages using *F.mean()*, with results immediately rounded to 5 decimal places.
- **Post-Aggregation Filtering and Ranking:** A subsequent filter retains only partitions where *num_flights* ≥ 500. The final ranking is achieved via *orderBy().desc().limit(10)*. Under the hood, Catalyst optimizes the limit(10) operator by executing a local limit on each partition before the final shuffle, drastically reducing the data transferred to the driver node.

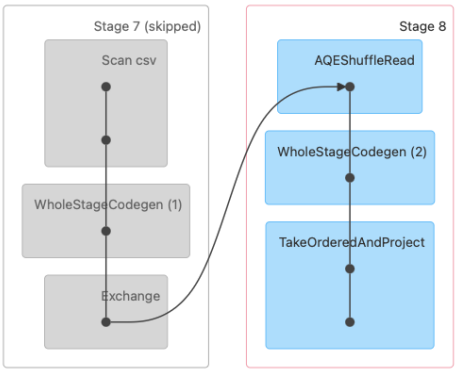


Fig. 6. Query 2 DataFrame

2) Spark SQL API Implementation: The SQL approach registers the underlying DataFrame as a temporary view and executes a purely declarative SQL string. The query logic mirrors the DataFrame approach using native SQL constructs, such as a *WHERE* clause for pre-filtering, *GROUP BY* for the aggregation, and *HAVING COUNT(*)* ≥ 500 to filter the final groups. Thanks to the Catalyst Optimizer, this SQL string is translated into the exact same physical execution plan as the DataFrame implementation. As a result, this approach achieves

perfect execution parity with DataFrames, offering the familiar readability of standard SQL while fully benefiting from the same underlying performance optimizations.

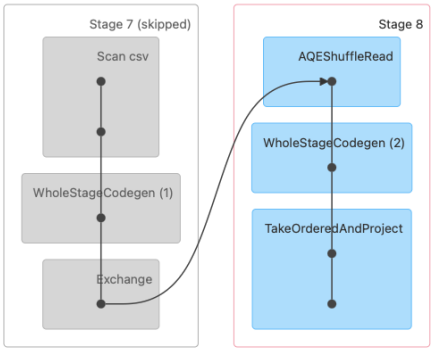


Fig. 7. Query 2 SQL

3) RDD API Implementation: Consistent with the imperative paradigm of the RDD API, this implementation requires the developer to explicitly define the map-reduce pipeline and manually orchestrate the aggregation arithmetic, completely bypassing the Catalyst Optimizer.

- **Row Parsing and Map-Side Filtering:** A custom *parse_row* function extracts the necessary fields, applying an immediate narrow filter (returning an empty list if a flight is canceled or diverted). Valid rows are mapped into complex key-value tuples: *(carrier, (1, arr, carr, wea, nas, sec, late))*.
- **Distributed Aggregation and Map-Side Combine:** The *reduceByKey()* wide transformation sums the occurrences and all six delay components component-wise. As previously noted, *reduceByKey()* inherently performs a map-side combine, significantly optimizing the network shuffle payload.
- **Manual Averaging and Bounded Ranking:** The reduced RDD is filtered to enforce the 500-flight minimum. Since RDDs do not possess built-in average functions, a custom *map(calc_averages)* transformation is applied to divide the aggregated sums by the flight count and round the results. Finally, instead of performing a full distributed sort, the *.top()* action is invoked. This is a highly optimized Spark action that maintains a bounded priority queue of specified size on each executor locally, sending only the top elements per partition to the driver, which then merges them to produce the final result.

V. SERVING LAYER AND VISUALIZATION

To guarantee optimal data retrieval for the visualization front-end, the architecture employs a polyglot persistence strategy. Rather than relying on a single database, the serving layer routes the output of the analytical workloads to distinct storage engines based on the structural semantics of the data.

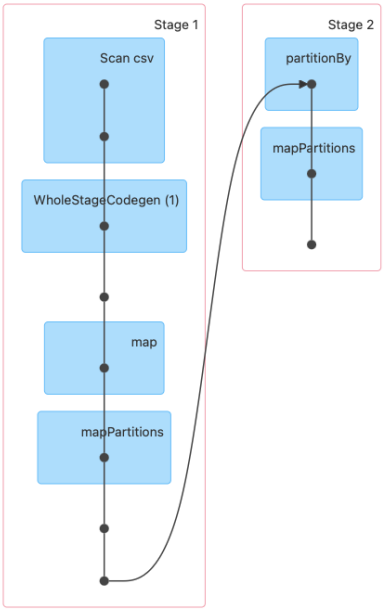


Fig. 8. Query 2 RDD

A. Time-Series Data Serving (InfluxDB)

InfluxDB, a high-performance time-series database, is designated for storing time-indexed metrics. It hosts two distinct flows of information:

- **Query 1 Analytical Results:** Since the first query aggregates delays and cancellation rates on a monthly basis, the output inherently represents a time-series. The custom *write_results_on_influxdb* Python function iterates over the Spark DataFrame rows, dynamically generating an artificial UTC timestamp for the first day of the corresponding month. The carrier codes are stored as indexed Tags, while the analytical results are stored as Fields.
- **Benchmark Metrics:** The *benchmark_runner.py* script automatically pushes the execution times of the analytical workloads to InfluxDB. Each entry is stored under a specific measurement representing the API/Format combination, and is tagged with the active cluster configuration (number of workers), establishing a historical baseline for the performance evaluation phase.

B. Key-Value Data Serving (Redis)

The results of the second query consist of a static, non-temporal ranking (the top 10 carriers by arrival delay and their delay components). Storing this leaderboard in a time-series database would be structurally inefficient. Consequently, these results are persisted in Redis. Before persistence, the analytical logic leverages the *.asDict()* PySpark method to convert the top 10 collected records into an array of dictionaries. This payload is then passed to the custom *write_results_on_redis* function, which serializes it into a JSON string and stored under a deterministic key mapping the specific run (e.g., *sabd:results:query2_csv_df*). This implementation guarantees

extremely low-latency read operations, perfectly matching the access patterns of live leaderboards.

C. Real-Time Visualization (Grafana)

Grafana acts as the unified observability platform, bridging the disparate datasources into a single, interactive dashboard composed of four specialized panels.

- **InfluxDB Integration (Query 1):** Two time-series panels utilize the Flux language to filter the data by measurement and field. They render line charts to visually compare the monthly trends of the mean departure delay and the cancellation rate between the targeted airlines.
- **Redis Integration (Query 2):** Grafana natively queries the Redis key-value store using the GET command. Because the payload is returned as a single serialized JSON array, a highly specific chain of built-in transformations is applied on the fly to render the charts: an initial *extractFields* parses the raw JSON string, a *transpose* operation pivots the array of objects into structured rows, a secondary *extractFields* decodes the nested JSON fields, and a final *filterFieldsByName* step isolates the required metrics. This pipeline elegantly maps the decoded data into a standard bar chart for the carrier ranking and a grouped bar chart to visualize the exact distribution of delay causes.

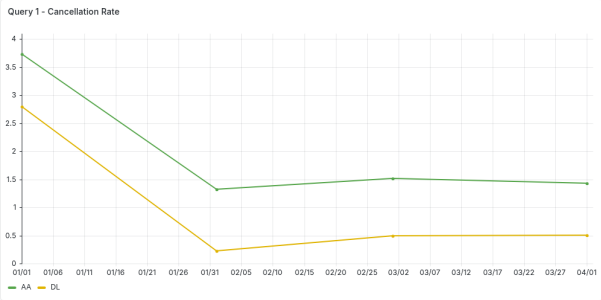


Fig. 9. Cancellation Rate

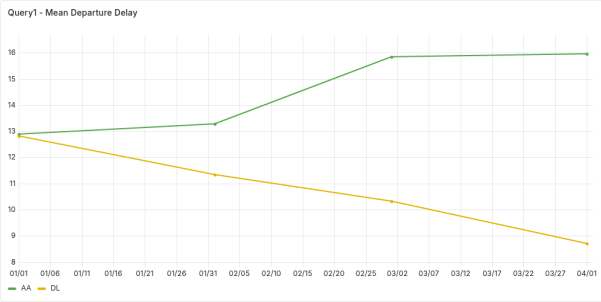


Fig. 10. Mean Departure Delay

VI. PERFORMANCE EVALUATION AND BENCHMARKING

To rigorously assess the efficiency of the developed architecture, a comprehensive performance evaluation was conducted.

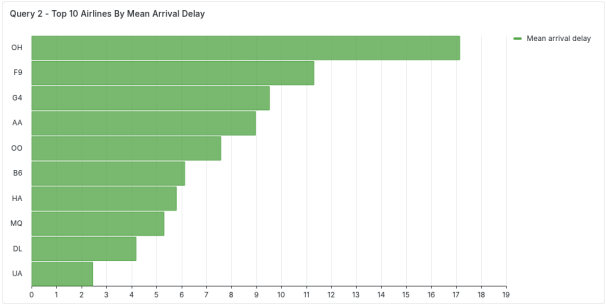


Fig. 11. Top 10 Airlines By Mean Arrival Delay

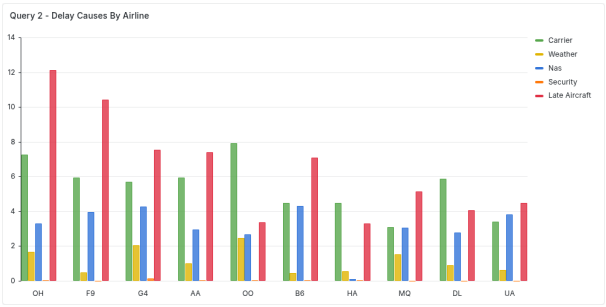


Fig. 12. Delay causes

The testing methodology focused on isolating the Spark computational engine by measuring the execution time of each query across varying API paradigms (RDD, DataFrame, Spark SQL), storage formats (CSV, Parquet), and degrees of horizontal scalability (from 1 to 4 Spark Workers).

A. Testing Methodology

The benchmarks were fully automated via a custom *benchmark_runner.py* script. To account for JVM cold starts, class loading, and initial resource allocation, each configuration was executed 10 times. The SparkSession was gracefully stopped and re-instantiated between iterations to prevent memory caching from artificially inflating the performance of subsequent runs. To ensure that the recorded execution times reflected purely the computational efficiency of the Spark engines, all I/O operations related to data persistence were explicitly bypassed during the benchmarking phase. Through a dedicated flag, the pipeline dynamically suppressed the exporting logic to HDFS, InfluxDB, and Redis. Instead, a simple *collect()* action was invoked to force the evaluation of the lazy transformations. Execution times were programmatically logged into InfluxDB for aggregation and visualized via Grafana.

B. Execution Time Analysis

This subsection presents the empirical data collected during the benchmarking phase. Tables I and II summarize the execution metrics for Query 1 and Query 2, respectively, detailing the minimum, maximum, mean, median, and variance across all iterations. It is worth noting that the maximum execution

times are inherently skewed by cold-start phenomena. Consequently, the mean and median values serve as the most reliable indicators of the system's stabilized computational speed.

TABLE I
QUERY 1 EXECUTION PERFORMANCE METRICS

API & Format	Workers	Min (s)	Max (s)	Mean (s)	Median (s)	Variance
DataFrame CSV	1	2.66	3.71	2.83	2.72	0.0905
	2	3.55	4.71	3.79	3.68	0.111
	3	4.13	4.99	4.32	4.25	0.0570
	4	4.97	5.67	5.15	5.09	0.0362
DataFrame Parquet	1	2.37	3.14	2.52	2.44	0.0454
	2	3.16	3.78	3.39	3.38	0.0256
	3	3.82	4.47	3.97	3.94	0.0312
	4	4.54	5.08	4.76	4.75	0.0251
RDD CSV	1	4.59	6.69	4.94	4.73	0.346
	2	5.23	6.49	5.51	5.41	0.121
	3	5.74	6.87	6.08	5.93	0.117
	4	6.65	7.54	6.99	6.99	0.0463
RDD Parquet	1	6.30	7.44	6.52	6.40	0.108
	2	7.03	7.79	7.35	7.38	0.0504
	3	7.38	8.16	7.73	7.79	0.0609
	4	8.18	8.85	8.54	8.58	0.0575
SQL CSV	1	2.61	3.60	2.78	2.70	0.0766
	2	3.51	4.52	3.75	3.67	0.0755
	3	4.13	5.00	4.32	4.27	0.0538
	4	4.94	5.85	5.17	5.13	0.0568
SQL Parquet	1	2.36	3.19	2.51	2.44	0.0545
	2	3.20	3.95	3.37	3.32	0.0444
	3	3.66	4.48	3.95	3.92	0.0548
	4	4.36	5.26	4.71	4.67	0.0508

TABLE II
QUERY 2 EXECUTION PERFORMANCE METRICS

API & Format	Workers	Min (s)	Max (s)	Mean (s)	Median (s)	Variance
DataFrame CSV	1	2.92	3.97	3.08	2.97	0.0913
	2	3.74	4.66	3.96	3.89	0.0579
	3	4.28	5.08	4.42	4.35	0.0528
	4	5.51	6.77	5.81	5.66	0.123
DataFrame Parquet	1	2.70	3.38	2.81	2.73	0.0383
	2	3.27	4.33	3.59	3.56	0.0982
	3	3.80	4.37	4.19	4.26	0.0311
	4	4.44	5.31	4.98	4.98	0.0614
RDD CSV	1	6.17	7.08	6.36	6.28	0.0613
	2	6.04	6.91	6.33	6.23	0.0641
	3	6.44	7.05	6.58	6.52	0.0315
	4	7.19	8.05	7.52	7.43	0.0639
RDD Parquet	1	9.64	10.30	9.85	9.83	0.0285
	2	10.20	11.70	10.70	10.60	0.134
	3	10.70	11.70	11.10	11.00	0.113
	4	12.40	12.70	12.50	12.50	0.00704
SQL CSV	1	2.90	3.86	3.08	2.99	0.0696
	2	3.77	4.78	3.91	3.81	0.0853
	3	4.25	5.12	4.47	4.35	0.0825
	4	5.53	6.28	5.73	5.67	0.0419
SQL Parquet	1	2.69	3.36	2.81	2.74	0.0355
	2	3.25	4.35	3.54	3.38	0.114
	3	3.76	4.41	4.06	4.08	0.0483
	4	4.48	5.38	4.95	4.98	0.0709

VII. DISCUSSION OF RESULTS

An in-depth analysis of the empirical data reveals several key insights regarding the underlying mechanics of the Spark execution engine:

A. Performance Equivalence of High-Level APIs:

The declarative programming paradigms (DataFrame and Spark SQL) exhibited a substantial performance advantage over the imperative RDD approach. For instance, in Query 2 (Parquet format, 1 Worker configuration), both the DataFrame and SQL implementations averaged 2.81 seconds, whereas the RDD equivalent required 9.85 seconds. This wide discrepancy highlights the critical role of Spark's internal optimizers (Catalyst optimizer) in dynamically generating highly efficient physical plans for higher-level abstractions. Furthermore, the near-identical execution times recorded for DataFrame and SQL consistently demonstrate that, regardless of the syntactic interface chosen by the developer, the underlying engine compiles both declarative approaches into the exact same optimized execution path.

B. Storage Format Implications:

When utilizing higher-level APIs, the columnar architecture of the Parquet format consistently outperformed traditional CSV files. Conversely, the benchmark exposed a severe performance penalty when processing Parquet files via the RDD API (e.g., in Query 1, Parquet averaged 6.52s compared to 4.94s for CSV). Consequently, when using low-level abstractions, parsing raw text formats proves to be empirically faster than interpreting optimized binary formats.

C. Horizontal Scalability and Network Overhead:

Scaling the computational cluster from one to four Spark Workers resulted in a monotonically worsening execution time across all test configurations. Given the relatively small size of the pre-processed dataset, the system exhibited clear negative scalability. In this context, the architectural overhead, specifically the latency introduced by task serialization, Master-Worker coordination, and data shuffling across the Docker virtual network, vastly eclipsed the theoretical performance gains of parallel computation.

D. System Tuning and Hardware Asymmetry

To optimize network throughput during wide transformations (e.g., shuffle operations), the default *spark.sql.shuffle.partitions* parameter, statically fixed at 200 by the Spark engine, was dynamically reconfigured at runtime. Preserving this default configuration for a dataset of reduced dimensions would have precipitated severe task overhead, forcing the Master node to schedule and dispatch hundreds of largely empty partitions across the virtual network. Adhering to Apache Spark's official tuning guidelines [9], which recommend allocating 2 to 3 tasks per available CPU core, the application was engineered to calculate the optimal partition count dynamically via the formula *Workers* × *Cores per Worker* × *2*. This adaptive scaling ensures that the physical execution plan remains strictly aligned with the underlying hardware topology, maximizing CPU utilization while eliminating the latency associated with over-partitioning.

Furthermore, executing the benchmarks locally on an ARM-based hybrid processor (Apple M4) exposed notable performance dynamics during horizontal scaling. Specifically, deploying three or four Spark Workers saturated the physical CPU, forcing the operating system to distribute identical computational tasks across both high-performance (P-cores) and high-efficiency (E-cores) units. Because Spark's symmetric execution model assigns equally sized data partitions to all available cores and requires all parallel tasks within a distributed stage to complete before proceeding, the overall

Map/Reduce execution time was systematically bottlenecked by the lower processing speeds of the efficiency cores. This empirical observation highlights the inherent architectural friction that occurs when deploying symmetric distributed processing frameworks atop asymmetric hardware topologies.

VIII. CONCLUSION

This project successfully deployed and evaluated a containerized Apache Spark architecture, demonstrating that efficient distributed data processing relies on a synergistic understanding of API abstractions, storage formats, and underlying hardware topologies.

Empirical benchmarking validated the performance equivalence of declarative APIs (DataFrame and Spark SQL), which systematically outperformed imperative RDDs thanks to the Catalyst Optimizer. Furthermore, while the Parquet format minimized disk I/O for higher-level APIs, it introduced severe deserialization overhead for RDDs, rendering raw CSV parsing empirically faster.

Finally, scaling experiments revealed clear negative scalability. For moderately sized datasets, the latency of cluster coordination and network shuffling vastly eclipses the theoretical gains of parallel computation. This was addressed by implementing dynamic runtime tuning for shuffle partitions to prevent over-partitioning. Additionally, local testing on an asymmetric ARM processor highlighted the architectural friction of symmetric distributed frameworks, as the overall Map/Reduce execution time was systematically bottlenecked by the lower processing speeds of the efficiency cores.

REFERENCES

[1] https://nifi.apache.org
[2] https://hadoop.apache.org/docs/r1.2.1/hdfs_design.html
[3] https://spark.apache.org
[4] https://www.influxdata.com
[5] https://redis.io
[6] https://grafana.com
[7] https://www.transtats.bts.gov/
[8] https://docs.docker.com/compose/
[9] https://spark.apache.org/docs/latest/tuning.html#level-of-parallelism